

HW3

Sri Srinivasan

July 2025

Problem 1

(a)

After batch normalization, the data is first normalized to have mean 0 and standard deviation 1, and then rescaled and shifted using the parameters $\gamma = 1$ and $\beta = 3$. Since $\gamma = 1$, the shape of the distribution remains the same, and $\beta = 3$ shifts the mean from 0 to 3. The resulting distribution is a Gaussian with mean 3 and standard deviation 1. The graph should therefore look like a standard bell curve centered at $x = 3$.

(b)

Batch normalization in 2D transforms the data so that it has mean at the origin and identity covariance. This means that the level sets after normalization are concentric circles centered at the origin. The 1-SD contour is a circle of radius 1, and the 2-SD contour is a circle of radius 2. The original ellipse becomes a unit circle.

Problem 2

(a)

Let $x(n)$ be the rectangular signal of length $L = 7$:

$$x(n) = \begin{cases} 1 & 0 \leq n \leq 6 \\ 0 & \text{otherwise} \end{cases}$$

Let $h(n) = \left(\frac{1}{2}\right)^n u(n)$:

$$h(n) = \begin{cases} \left(\frac{1}{2}\right)^n & n \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

We find the convolution

$$y(n) = (x * h)(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k)$$

As $x(k)$ is nonzero only for $k = 0$ to 6 , and $h(n-k)$ is nonzero for $n-k \geq 0$, the limits are now

$$y(n) = \sum_{k=0}^6 x(k)h(n-k) = \sum_{k=0}^6 h(n-k).$$

We find $y(n)$ for $n = -6$ to 12 .

n	$y(n)$
-6 to -1	0
0	$h(0) = 1$
1	$h(0) + h(1) = 1 + \frac{1}{2}$
2	$h(0) + h(1) + h(2) = 1 + \frac{1}{2} + \frac{1}{4}$
3	$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8}$
4	$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16}$
5	$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32}$
6	$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{64}$
7	$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{64}$
8	$\frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{64}$
9	$\frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{64}$
10	$\frac{1}{16} + \frac{1}{32} + \frac{1}{64}$
11	$\frac{1}{32} + \frac{1}{64}$
12	$\frac{1}{64}$

(b)

Define $x_2(n) = x(n-5)$. This shifts the rectangular window to the right by 5 units.

Then,

$$y_2(n) = (x_2 * h)(n) = \sum_{k=-\infty}^{\infty} x_2(k)h(n-k) = \sum_{k=-\infty}^{\infty} x(k-5)h(n-k).$$

By change of variable $k' = k - 5$, $k = k' + 5$,

$$y_2(n) = \sum_{k'=-\infty}^{\infty} x(k')h(n-k'-5) = \sum_{k'=-\infty}^{\infty} x(k')h((n-5)-k') = y(n-5).$$

Thus, the property is shifting the input shifts the output.

(c)

Given:

$$x = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}, \quad h = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Use convolution:

$$y[m, n] = \sum_{i, j} x[m - i, n - j] h[i, j]$$

Evaluating at (1, 1), we have

$$\text{Region in } x : \begin{bmatrix} 1 & 2 & 3 \\ 6 & 7 & 8 \\ 11 & 12 & 13 \end{bmatrix}$$

$$y[1, 1] = (-1)(1) + (-2)(2) + (-1)(3) + 0 + 0 + 0 + (1)(11) + (2)(12) + (1)(13) = -1 - 4 - 3 + 0 + 0 + 0 + 11 + 24 + 13 = 40$$

(d)

- (i) Stride 1, Pad 1: Padding adds 1 row/col of zeros on each side. Input becomes 7×7 . Output size = 5×5 .
- (ii) Stride 2, Pad 1: Same padded input. Output size:

$$\left\lfloor \frac{7-3}{2} \right\rfloor + 1 = 3 \Rightarrow 3 \times 3 \text{ output}$$

Each case follows the same convolution method using padded x and strided movement of the window.

Problem 3

(a)

- Which one denotes the process of batch normalization? C
- Which one denotes layer normalization? A

(b)

Given:

$$\bar{x}_i = x_i - \mu \quad \text{where} \quad \mu = \frac{1}{n} \sum_{j=1}^n x_j$$

$$y_i = \gamma \bar{x}_i + \beta$$

We are to find

$$\frac{\partial L}{\partial x_i} \quad \text{in terms of} \quad \frac{\partial L}{\partial y_j}$$

Let

$$\frac{\partial L}{\partial y_j} = g_j$$

First, we have

$$\frac{\partial y_j}{\partial x_i} = \gamma \cdot \frac{\partial \bar{x}_j}{\partial x_i}$$

and,

$$\bar{x}_j = x_j - \mu = x_j - \frac{1}{n} \sum_{k=1}^n x_k \Rightarrow \frac{\partial \bar{x}_j}{\partial x_i} = \delta_{ij} - \frac{1}{n}$$

$$\frac{\partial y_j}{\partial x_i} = \gamma \left(\delta_{ij} - \frac{1}{n} \right)$$

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^n \frac{\partial L}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_i} = \sum_{j=1}^n g_j \cdot \gamma \left(\delta_{ij} - \frac{1}{n} \right) = \gamma \left(g_i - \frac{1}{n} \sum_{j=1}^n g_j \right)$$

$$\boxed{\frac{\partial L}{\partial x_i} = \gamma \left(\frac{\partial L}{\partial y_i} - \frac{1}{n} \sum_{j=1}^n \frac{\partial L}{\partial y_j} \right)}$$

If $n = 1$, then:

$$\mu = x_1 \Rightarrow \bar{x}_1 = 0 \Rightarrow y_1 = \beta \Rightarrow \frac{\partial y_1}{\partial x_1} = 0 \Rightarrow \frac{\partial L}{\partial x_1} = \gamma(0 - 0) = 0$$

$\square$

As $n \rightarrow \infty$, the term

$$\frac{1}{n} \sum_{j=1}^n \frac{\partial L}{\partial y_j} \rightarrow \text{mean of gradients} = \bar{g}$$

Therefore,

$$\frac{\partial L}{\partial x_i} = \gamma \left(\frac{\partial L}{\partial y_i} - \bar{g} \right)$$

This shows that the gradient becomes the difference between the local gradient and the mean gradient, scaled by γ .

Problem 4

(a)

The number of weights in a convolutional layer is:

$$\text{Weights} = K \times K \times C \times F$$

The number of biases is F (one per filter). So, total parameters is

$$\text{Parameters} = K^2 \cdot C \cdot F + F$$

The output feature shape is:

$$W' = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1, \quad H' = \left\lfloor \frac{H - K + 2P}{S} \right\rfloor + 1, \quad C' = F$$

(b)

Max pooling with kernel size K and stride S gives:

$$W' = \left\lfloor \frac{W - K}{S} \right\rfloor + 1, \quad H' = \left\lfloor \frac{H - K}{S} \right\rfloor + 1, \quad C' = C$$

(c)

If there are L successive convolutional layers, each with stride 1 and kernel size K , then the receptive field grows as:

$$R = 1 + (K - 1) \cdot L$$

Each convolution increases the receptive field by $K - 1$ due to overlapping.

(d)

If a max pooling layer has kernel size 2 and stride 2, it downsamples by a factor of 2, the receptive field doubles, because each output now sees a 2x2 area of the input.

An advantage is that it reduces spatial dimensions, but it also loses spatial precision.

(e)

Given:

- CONV3-10: 3×3 filters, 10 filters, input depth C , padding 1, stride 1.
- POOL2: 2×2 max-pool, stride 2.

Layer	Number of Parameters	Dimension of Activations
Input	0	$28 \times 28 \times 1$
CONV3-10	$3 \times 3 \times 1 \times 10 + 10 = 100$	$28 \times 28 \times 10$
POOL2	0	$14 \times 14 \times 10$
CONV3-10	$3 \times 3 \times 10 \times 10 + 10 = 910$	$14 \times 14 \times 10$
POOL2	0	$7 \times 7 \times 10$
FLATTEN	0	490
FC-3	$490 \times 3 + 3 = 1473$	3

(f)

We are told:

$$\mathbf{g}_1 = f(\mathbf{x}_1) = \begin{bmatrix} 0.8 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{g}_2 = f(\mathbf{x}_2) = \begin{bmatrix} 0 \\ 0.8 \\ 0 \end{bmatrix}$$

We have - $\mathbf{x}_1$ is a horizontal edge $\Rightarrow$ activates the first channel, and $\mathbf{x}_2$ is a vertical edge $\Rightarrow$ activates the second channel.

Now,

$$\mathbf{x}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ \vdots & & & & & & & \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \Rightarrow \text{vertical edge on the right} \Rightarrow \boxed{\mathbf{g}_3 = \mathbf{g}_2 = \begin{bmatrix} 0 \\ 0.8 \\ 0 \end{bmatrix}}$$

$$\mathbf{x}_4 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \vdots & & & & & & \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \Rightarrow \text{horizontal edge on bottom} \Rightarrow \boxed{\mathbf{g}_4 = \mathbf{g}_1 = \begin{bmatrix} 0.8 \\ 0 \\ 0 \end{bmatrix}}$$

Problem 6

Q1

After training on the small dataset, the filters learned by the CNN bear often focus on detecting oriented gradients, like vertical or horizontal edges, which makes sense given the nature of the dataset. The patterns show that even with minimal data, CNNs can learn low-level features.

Q2

To understand how inductive bias manifests in CNNs for this part of the problem, we compared three types of kernels from the first convolutional layer.

1. Learned kernels: weights obtained after training a CNN on an edge detection task.
2. Untrained kernels: initial random weights before training.
3. Edge-detector kernels: Sobel filters to detect horizontal and vertical edges.

Untrained kernels appeared visually noisy and lacked meaningful spatial structure. They did not look like edge detectors.

Learned kernels, when trained with a small dataset (10 images per class), similarly failed to have edge-detection patterns. Their appearance remained somewhat amorphous, meaning that the model struggled to learn meaningful features in the low-data part.

In contrast, the edge-detector kernels initialized using Sobel filters showed distinct, directional structures. When the CNN was initialized with Sobel filters, it achieved significantly higher validation accuracy (64%) compared to both the randomly initialized CNN (30%) and the MLP (41.33%) on the same data.

This demonstrates that while CNNs have an inductive bias toward locality and translation invariance, this bias may not manifest effectively without sufficient data or well-done initialization.

In the other experiment in Q2, we froze the convolutional layer after initializing it with edge detector kernels and trained only the classifier (final layer). We observed that in the low-data part, this improved performance. However, in the high-data part, it underperformed compared to a randomly initialized CNN with trainable filters.

This demonstrates that fixed inductive bias (edge detectors) helps when in low-data but limits flexibility when there is sufficient data to learn better features.

Q3

We trained both the CNN and the MLP on 30 training images per class and looked at their training and validation performance across epochs. Initially, we used 30 epochs but saw that the validation accuracy and loss were not flat toward the end, showing that the model had yet to converge.

Epochs trained: 30, 60, 90, 120

At 120 epochs, the CNN's validation accuracy and loss curves stabilized, suggesting convergence.

Final CNN accuracy: 95.33% (on validation set with 30 images per class)

After training the CNN to convergence, we visualized the learned convolutional kernels. The filters clearly exhibited edge-like structures, resembling edge detectors. This demonstrates that, when trained on sufficient data and given enough training time, the CNN can understand low-level visual features that are necessary for edge detection.

We next understood how much data is needed for the MLP to match the performance of the CNN. For each number of training images per class, we trained both models and compared their final validation accuracy.

CNNs achieve high performance (above 95%) with only 30 training images per class. In contrast, MLPs require at least 100 training images per class to achieve a comparable level of accuracy.

This supports the idea that CNNs, due to their inductive biases (locality and translation invariance), are more data-efficient than MLPs. CNNs can generalize well even with limited data, while MLPs require many more examples to reach similar performance (approximately $3\text{--}4\times$ more training data).

Q4

Since training images contain edges in consistent locations, CNNs may overfit to these positions. During validation, if edges appear in different parts of the image, the CNN may struggle. MLPs do not exploit spatial location and treat all pixels uniformly, allowing it to generalize more uniformly across edge positions.

Q5

Permuting the pixel order breaks spatial structure in the input, which CNNs depend on to detect local patterns. As a result, CNNs fail to learn in this case.

MLPs, which do not rely on spatial locality, are unaffected by such permutations and thus maintain their ability to learn.

Increasing the convolutional kernel size in CNNs causes validation accuracy to degrade. Larger kernels reduce the locality of the filters and make the network look like a fully connected one, which weakens the benefits of the convolutional model.

Q6

As the number of classes increases, max pooling may start to underperform compared to average pooling. Average pooling provides a more holistic summary of the feature map compared to max pooling.

Problem 7

(a)

We have:

$$y_{i,j} = \sum_{h=1}^k \sum_{l=1}^k x_{i+h-1,j+l-1} \cdot w_{h,l}$$

So,

$$\frac{\partial \mathcal{L}}{\partial w_{h,l}} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial y_{i,j}} \cdot \frac{\partial y_{i,j}}{\partial w_{h,l}} = \sum_{i,j} dy_{i,j} \cdot x_{i+h-1,j+l-1}$$

Thus, each entry of $d\mathbf{W}$ is:

$$(d\mathbf{W})_{h,l} = \sum_{i,j} dy_{i,j} \cdot x_{i+h-1,j+l-1}$$

For SGD with learning rate η , the update is:

$$w_{h,l} \leftarrow w_{h,l} - \eta \cdot (d\mathbf{W})_{h,l}$$

(b)

Assume $x_{i,j}$ and $dy_{i,j}$ are i.i.d. with:

$$E[x_{i,j}] = 0, \quad E[dy_{i,j}] = 0, \quad \text{Var}(x_{i,j}) = \sigma_x^2, \quad \text{Var}(dy_{i,j}) = \sigma_y^2$$

From part (a), each entry of $d\mathbf{W}$ is:

$$d\mathbf{W}_{h,l} = \sum_{i,j} dy_{i,j} \cdot x_{i+h-1,j+l-1}$$

Then:

$$E[d\mathbf{W}_{h,l}] = \sum_{i,j} E[dy_{i,j}] \cdot E[x_{i+h-1,j+l-1}] = 0$$

$$\text{Var}(d\mathbf{W}_{h,l}) = \sum_{i,j} \text{Var}(dy_{i,j} \cdot x_{i+h-1,j+l-1}) = \sum_{i,j} \sigma_y^2 \cdot \sigma_x^2$$

The output matrix Y has shape $(n - k + 1) \times (n - k + 1)$, so:

$$\text{Var}(d\mathbf{W}_{h,l}) = (n - k + 1)^2 \cdot \sigma_x^2 \cdot \sigma_y^2$$

$$\text{StdDev} = \sqrt{\text{Var}} = (n - k + 1) \cdot \sigma_x \cdot \sigma_y$$

Thus, the standard deviation grows linearly with image size n .

(c)

Let

$$X = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix}, \quad y_{1,1} = \max(x_{11}, x_{12}, x_{21}, x_{22})$$

Max pooling: Assume upstream gradient is $dy_{1,1}$. Let (i, j) be the index where $x_{ij} = y_{1,1}$ and is unique. Then:

$$dX_{i,j} = dy_{1,1}, \quad \text{all other entries } 0$$

e.g., if x_{21} is max:

$$dX = \begin{bmatrix} 0 & 0 \\ dy_{1,1} & 0 \end{bmatrix}$$

Average pooling: Since

$$y_{1,1} = \frac{1}{4}(x_{11} + x_{12} + x_{21} + x_{22}), \quad \Rightarrow \quad \frac{\partial y_{1,1}}{\partial x_{i,j}} = \frac{1}{4}$$

So:

$$dX = \frac{dy_{1,1}}{4} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

(d)

Max pooling:

- Picks the strongest feature in each region.
- Introduces sparsity, which improves generalization.
- Issue is that it completely ignores weaker features.

Average pooling:

- Retains all features by averaging
- Helps gradient propagate more evenly during backprop.
- Issue is that it can blur strong features.